Harmony and Operations

Ke Liang

Capstone

April 16, 2025

Introduction

Given that music's initial scales were mathematically formulated and subsequent research frequently uncovered mathematical structures within music, it seems plausible that mathematics plays an essential role in shaping musical harmony. This association suggests that mathematical patterns and relationships may explain harmony in theoretical terms. Bach (and many other musicians) had long been described as a musician as well as a mathematician, his musical constructions, like Crab Canon, was constructed to be played on a Mobius strip. Considering these ingenious crafts, it's natural to wonder, what kinds of music is harmonious? what kind of music remains harmonious after certain operations like being played on a Mobius strip? Digging into these questions is important not only because it sounds inspirational, but also because characterizing the operations sheds some implication in generating music out of old ones, and might shape how AI generate interesting, harmonic music pieces. On a even larger scale, it potentially reveal a subtle homomorphic relation between art and science.

Literature Review

The mathematical exploration of music has deep historical roots, dating back to Pythagoras's observations on string lengths and harmonic ratios. In Musimathics, Gareth Loy underscores how these ancient discoveries have evolved into a comprehensive mathematical framework for understanding acoustics, pitch, scales, and musical form. Loy emphasizes the foundational role of information theory, Fourier analysis, and Markov processes in modeling listener expectations and musical structure—core tools directly used in this paper's definition

of harmony through entropy.

Building on these technical foundations, Guerino Mazzola's The Topos of Music takes a radically abstract approach, introducing category theory and topos theory to formulate a conceptual ontology of music. Mazzola interprets musical elements as mathematical objects within structured logical universes, where symmetry, transformations, and morphisms become ways of understanding music's internal logic. His treatment of musical morphisms and denotators supports this paper's interest in how operations like retrogradation affect musical identity. Mazzola's framework, especially the functorial perspective on transformations, helps formalize the idea that some musical operations may preserve entropy or consonance, while others disrupt it.

Another essential influence is Generalized Voice-Leading Spaces by Callender, Quinn, and Tymoczko, which provides a geometric framework for modeling voice leading as points in high-dimensional space. Their concept of musical "distance" based on minimal movement in pitch space complements this paper's use of directed graphs to model melodic transitions. Voice-leading spaces can be seen as underlying manifolds that, when paired with entropy-based analysis, offer a unique lens into how predictable—or surprising—musical movement appears.

Other studies further illuminate this relationship between expectation and structure. For instance, David Huron's work on Sweet Anticipation builds on psychological studies of

musical expectation using information theory, showing how listeners cognitively engage with probabilistic models of pitch and rhythm. This supports the core assumption in this paper: that listener surprisal, quantifiable via entropy, plays a central role in defining harmony.
On the consonance side, research in computational music theory—such as that by Sethares (on adaptive tuning systems) or neural net-based models of chord perception—has shown how consonance judgments can be both learned and modeled algorithmically. The present paper contributes to this domain by training a neural network to extend consonance estimation to more complex chords, addressing the gap in assigning fixed consonance values beyond dyads.

Overall, this body of work provides a strong interdisciplinary foundation for the study of musical harmony as a mathematical object. Loy gives us the acoustical and probabilistic groundwork, Mazzola supplies an ontological and transformational system, and Callender et al. offer a geometric intuition. Together, these sources support a new approach to visualizing and quantifying harmony—not merely as an artistic intuition but as a structured phenomenon governed by mathematical laws.

Theoretical Framework: Definition of Harmony

Before talking about harmony, we first need to define harmony. This paper defines harmony through two indices: information entropy and consonance.

Harmony includes information entropy because whether a piece of music is harmonious

highly depends on audiences' expectations. When a phrase of melody continue to repeat, audiences get bored, but when a melody continues to vary without pattern, it sounds chaotic and lacks meaning. Therefore, a harmonious piece of music controls audiences' surprisal within a reasonable range, we define musical information entropy accordingly.

Suppose you are predicting the next note in a given melody, you can predict that the next note belongs to the upper half of a piano, if it comes out indeed, you will be happily surprised, and thus record a 1 (otherwise 0) to represent your surprisal. The amount of surprisal requires one binary digit, or one bit, to represent. If you take a higher risk and predict that the next note belongs to the bottom quarter of a piano, the probability of you guessing correctly is $\frac{1}{4}$. Because it's less likely, when the next note comes out exactly as you predicted, you will be twice as surprised as the previous scenario, at the mean time, your surprisal requires two binary digits, 2^2, to represent a magnitude of four outcomes.

Thus we deduce $p = \frac{1}{2^s}$, where p is the probability of guessing correctly and s represents surprisal.

$$s = -\log_2 p = -\frac{\ln p}{\ln 2}$$

Note that this definition for surprisal preserves its additive nature, so that when doing consecutive predictions, the probability experiences multiplication while surprisal can be added.

Now suppose there's a piece of music with in total N notes, the ith key is played N_i times,

then the average surprisal of this piece of music would be:

$$H(X) = \sum_{i=1}^{88} \frac{N_i}{N} s_i$$

when N approaches infinity, $\frac{N_i}{N}$ approaches p_i,

$$\sum_{i=1}^{88} \frac{N_i}{N} s_i = \sum_{i=1}^{88} p_i s_i = -\sum_{i=1}^{88} p_i \log_2 p_i = -k \sum_{i=1}^{88} p_i \ln p_i$$

Note that this expression aligns with the official definition of Shannon information entropy in physics:

$$H(X) = -K \sum_{i=1}^{M} p_i \ln p_i$$

In the following paper for the sake of convenience, we will take $H(X) = -\sum_{i=1}^{M} p_i \ln p_i$ to analyze properties of music structures. But what is the total entropy of a piece of music? There is entropy disregarding a pitch's relation to other pitches but solely consider the probability of that pitch occurring itself, but also the probability of a permutation of pitch repeated. For example, if the piece of music comprises a phrase E-E-G-C, then, we not only consider the probability of C, E, G occurring, but also consider the probability of E-E, E-G, G-C (provided the first note, the probability of the second note occurring), and E-E-G, E-G-C occurring. We use Markov's chain to analyze the probability of each event happening.

For the previous example, the zeroth order probability:

C	D	E	F	G	A	B

Probability	1/4	0	1/2	0	1/4	0	0

First order probability:

Y \ X	C	D	E	F	G	A	B
C	0	0	0	0	1	0	0
D	0	0	0	0	0	0	0
E	0	0	1/2	0	0	0	0
F	0	0	0	0	0	0	0
G	0	0	1/2	0	0	0	0
A	0	0	0	0	0	0	0
B	0	0	0	0	0	0	0

whereas the probability is $P(X|Y)$.

Third order:

Y \ X	E-E	E-G	G-C
C	0	1	0
D	0	0	0
E	0	0	0
F	0	0	0

G	1	0	0
A	0	0	0
B	0	0	0

As the order continues to increase, the pitch permutation becomes more and more defined, the probability of each permutation approaches solely 0 and 1, and the surprisal they bring approaches 0. With the probability of a music piece defined for each order, we can thus define the entropy for each order and take summation to get the total entropy.

Other than entropy, our definition of harmony also includes consonance.

Intervals (semitone)	Interval name	Consonance rank
0 or 12	unison/octave	1
1	minor second	11
2	major second	8
3	minor third	6
4	major third	4
5	perfect fourth	3
6	augmented fourth/diminished fifth	12
7	perfect fifth	2
8	minor sixth	7
9	major sixth	5
10	minor seventh	9
11	major seventh	10

(https://www.researchgate.net/figure/The-mapping-rule-for-pitch-interval-to-the-consonance-rank_fig5_283543167)

According to this ranking, we can assign initial consonance scores for each dyads chords, the neural network first randomly generates weights to connections between neurons and tries to predict consonance scores for each interval. It compares its predictions to the actual scores

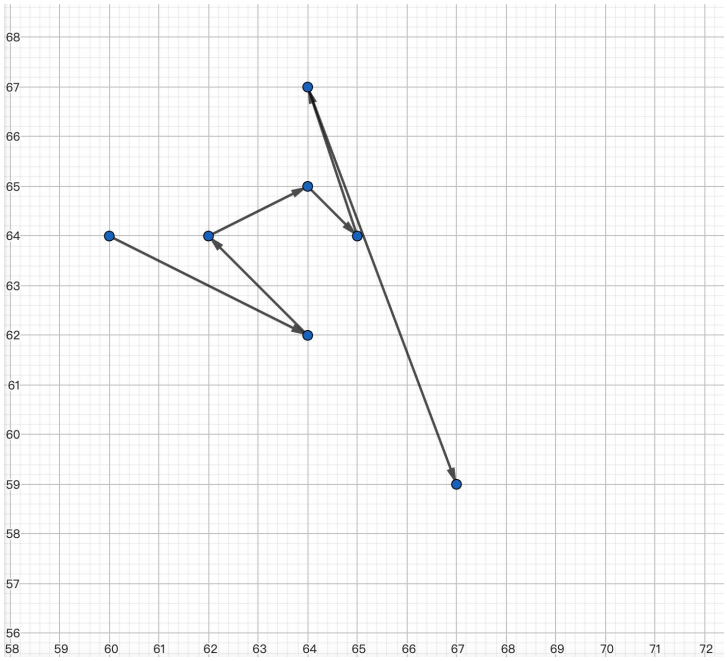
we assigned and updates its weights to reduce the error, using backpropagation. The process repeats for many times (epochs) until its predictions match the initial table as closely as possible. Through this network, we avoid manually and subjectively assigning scores for more complicated chords, like chords with up to five notes or so. It turns out that the prediction, or to say the relative consonance that we can subjectively estimate, is generally accurate.

The python code written for average information entropy and consonance score calculation is attached in appendix (Appendix 1).

Theoretical Framework: Visualizing Connections

After defining harmonious, I would like to introduce a way to visualize connections in a music score. Through the method introduced in "Generalized Voice-Leading Spaces" by Callender, Quinn, and Tymoczko, every musical object can be presented by a point in $\mathbb{R}^n$. In this paper, we will take this music object as the previously mentioned permutation of pitches, single note can be represented by points on a one-dimension number line, while note transitions like E-G can be represented by a point on $\mathbb{R}^2$. This paper primarily treats each of the 88 pitches on a keyboard differently, so the point representing note transitions would be defined by notes' MIDI numbers. For example, C4-E4 transition can be represented by coordinate $(60,64)$, where 60 is the MIDI number for C4 and 64 is the MIDI number for E4. After plotting the points, one can connect the points in the order of melodious propagation and form a directed graph with coordinates. For example, for melody

C4-E4-D4-E4-F4-E4-G4-A3, the directed graph looks like the following:



Result and Discussion: About Graphs

At first we are going to manipulate the entropy formula for the graph setting. Let's consider the first order entropy. First order entropy corresponds to a graph on $\mathbb{R}^2$, where the x-coordinate of a point represents the given pitch, and the y-coordinate represents the pitch after that.

Note that for a vertex that's not the starting or ending point of the graph, there will be same number of edges pointing towards and away from the vertex, and the number of edges that points towards the vertex is equal to the number of times that this pitch permutation occurs. Suppose there's $\mu_{(i,j)}$ times that pitch permutation (i, j) occurs, $\mu_{(i,j)} = \frac{E_{(i,j)}}{2}$. For the starting and ending vertex, there's a single edge pointing towards or away from the vertex that's unpaired, but we can imagine it being as a part of another continuous graph and manually

pair it up. So $\Gamma\mu_{(i,j)} = \frac{E_{(i,j)}}{2}\rfloor$. Now suppose for $\mu_i = \sum_j \mu_{(i,j)}$ times that a pitch permutation starts with pitch i, we notice that $\frac{\mu_{(i,j)}}{\mu_i} = P(i \mid j)$. Thus,

$$H(X) = -\sum_i \sum_j \frac{\mu_{(i,j)}}{\mu_i} ln \frac{\mu_{(i,j)}}{\mu_i}$$

let $F = \frac{\mu_{(i,j)}}{\mu_i} ln \frac{\mu_{(i,j)}}{\mu_i}$, notice:

$$\mu_{(i,j)} = 0 \Rightarrow F = 0$$

$$\mu_{(i,j)} = 1 \Rightarrow F = \frac{1}{\mu_i} ln \frac{1}{\mu_i}$$

$$\mu_{(i,j)} = z, z \geq 2 \Rightarrow F = \frac{z}{\mu_i}(ln\, z - ln \mu_i)$$

About graphs generated for note transitions, there are three observations. First, simple graph corresponds to the largest entropy under that vertex set. That is for example, fig.1 corresponds to larger entropy than fig.2.

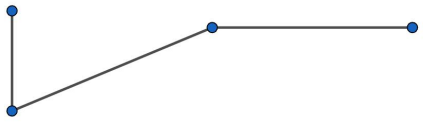


fig.1

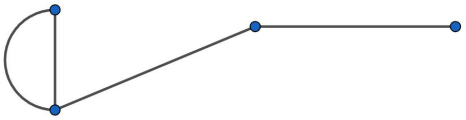


fig2

This conclusion very much aligns with our intuition, because the more a pitch permutation

appears, the less surprisal it brings us. Mathematically, it can be proven as following:

In a simple graph, for each x-coordinate *i*, we know that $\mu_{(i,j)} = 1$,

$$-\mathrm{H_i(X)} = \sum_j \frac{\mu_{(i,j)}}{\mu_i} ln \frac{\mu_{(i,j)}}{\mu_i} = \sum_j \frac{1}{\mu_i} ln \frac{1}{\mu_i}$$

the summation is independent from μ_i. Because we are doing the summation for a certain *i*, μ_i is constant,

$$\sum_j \frac{1}{\mu_i} ln \frac{1}{\mu_i} = ln \frac{1}{\mu_i} \sum_j \frac{1}{\mu_i} = \frac{1}{\mu_i} ln \frac{1}{\mu_i} \sum_j \mu_{(i,j)}$$

by definition $\sum_j \mu_{(i,j)} = \mu_i$, so we get $-H_i(X) = ln \frac{1}{\mu_i}$.

Now consider the non-simple graph, for every given *i*, by definition,

$$-\mathrm{H_i'(X)} = \sum_j \frac{\mu_{(i,j)}}{\mu_i} ln \frac{\mu_{(i,j)}}{\mu_i}$$

we know that $xlnx$ is a convex function, so we can apply Jensen inequality:

$$\mathrm{f}(\frac{\mathrm{x_1 + x_2 + \ldots + x_n}}{\mathrm{n}}) \leq \frac{\mathrm{f(x_1) + f(x_2) + \ldots + f(x_n)}}{\mathrm{n}}$$

Define $f(x) = xlnx$let $\mathrm{x_j} = \frac{\mu_{(i,j)}}{\mu_i}$, then n is equal to the number of vertex with x-coordinate *i*.

$$\sum_j \frac{\mu_{(i,j)}}{\mu_i} = 1$$

$$\mathrm{f}(\frac{\mathrm{x_1 + x_2 + \ldots + x_n}}{\mathrm{n}}) = \frac{1}{\mathrm{n}} \mathrm{ln} \frac{1}{\mathrm{n}}$$

$$\frac{\mathrm{f(x_1) + f(x_2) + \ldots + f(x_n)}}{\mathrm{n}} = -\frac{1}{\mathrm{n}} \mathrm{H_i'(X)} = \frac{1}{\mathrm{n}} \sum_j \frac{\mu_{(i,j)}}{\mu_i} ln \frac{\mu_{(i,j)}}{\mu_i}$$

$$\mathrm{ln} \frac{1}{\mathrm{n}} \leq \sum_j \frac{\mu_{(i,j)}}{\mu_i} ln \frac{\mu_{(i,j)}}{\mu_i}$$

notice that for a simple graph, every point is traversed only once, so $\mu_i = n$,

$$\ln\frac{1}{\mu_i} \leq \sum_j \frac{\mu_{(i,j)}}{\mu_i} ln \frac{\mu_{(i,j)}}{\mu_i}$$

now take both sides negative to get $H(X)$ and $H'(X)$, we get $H_i(X) \geq H_i'(X), \forall i \in \mathbb{Z}^+$. This might be counter-intuitive in visual sense, because we generally thinks more edges represents more information in a graph, however, under this definition, the opposite holds true.

Second, take two parts of graphs (or 2 different x-coordinates), suppose μ_i are equal, and the two parts corresponds to the same $\sum \mu_{(i,j)}$(not necessarily μ_i), the entropy of a more edges distributed graph is larger than entropy of a one-vertex concentrated graph.

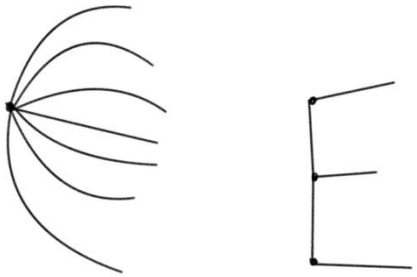


fig.1

fig.2

that's saying, fig.1 corresponds to less entropy than fig.2.

Say, for $\mu_{(i,j)}$ in two parts of graphs with not necessarily same constant i, re-index it as z_j, we know $\sum z_j = R$, since μ_i is constant,

$$\sum_{j=1}^{n} z_j = \sum_{j=1}^{1} z'_j = \mu_i = c$$

$$F_2 = \sum_{j=1}^{n} \frac{z_j}{c}(\ln z_j - \ln c) = \frac{1}{c}\sum_{j=1}^{n} z_j (\ln z_j - \ln c) = \frac{1}{c}(\sum_{j=1}^{n} z_j \ln z_j - R \ln c)$$

similarly,

$$F_1 = \frac{R}{c}\ln\frac{R}{c} = \frac{R}{c}(\ln R - \ln c)$$

$$c(F_1 - F_2) = R\ln R - \sum_{j=1}^{n} z_j \ln z_j$$

to determine whether this is positive, we can compare there exponential form:

$$R^R - \prod_{j=1}^{n} z_j^{z_j}$$

WLOG, assume z_n is the maximum in set $\{z_j | \ 1 \le j \le n\}$,

$$\prod_{j=1}^{n} z_j^{z_j} \le z_n^{R} < R^R$$

thus,

$$F_1 > F_2, -F_1 < -F_2, H_1 < H_2$$

Third, if two parts of a graph (or 2 different x-coordinates), with the same number of vertices, and the two parts corresponds to the same $\sum \mu_{(i,j)}$(not necessarily μ_i) and the same μ_i, then the more concentrated the edges are, the less entropy it corresponds to.



fig. 1



fig. 2

That's saying, fig.1 corresponds to less entropy than fig.2.

Say, for $\mu_{(i,j)}$ in two parts of graphs with not necessarily constant i, re-index it as z_j. Since μ_i is constant, let $\mu_i = c$. Consider

$$\sum_{j=1}^{n} z_j = R$$

$$F = \sum_{j=1}^{n} \frac{z_j}{c}(\ln z_j - \ln c) = \frac{1}{c}\sum_{j=1}^{n} z_j(\ln z_j - \ln c) = \frac{1}{c}(\sum_{j=1}^{n} z_j \ln z_j - R \ln c)$$

We notice that take $f(x) = x \ln x$, the function is convex, and $f(x) \geq 0$ when $x \geq 1$, this implies that in our case

$$f(x + a + 1) \geq f(x + 1) - f(x) > f(x) - f(x - 1), a \geq 0$$

thus taking any pair of numbers $(d, e) \in \{z'_j | 1 \leq j \leq n\}$, without loss of generality, assume $d \leq e$, then

$$f(d - 1) + f(e + 1) > f(d) + f(e)$$

repeating the operation until $z_1 = z_2 = \ldots = z_{m-1} = 1$ and $z_m = c - m + 1$, then $\sum_{j=1}^{m} z_j \ln z_j$ reaches its maximum. So

$$\sum_{j=1}^{m} z_j \ln z_j \leq (c - m + 1)\ln(c - m + 1)$$

Therefore, $F \leq (c - m + 1)\ln(c - m + 1), H = -F \geq -(c - m + 1)\ln(c - m + 1)$.

Through our reasoning about convexity, we can also deduce that the more concentrated z_j is, the less entropy it corresponds to.

These three observation might sound counter-intuitive because visually, comparing a simple graph and a non-simple graph, an averaged out graph and a less distributed graph, we naturally feel chaos in the latter ones. However, they actually encode less information entropy. Like wave function in physics, a flat line wave seems like the most regular, clean and predictable, but it encodes the most uncertainty and possibilities.

Result and Discussion: About Operations

The primary operation that we will talk about in this section is retrogradation, by retrogradation we mean playing a score completely backward, starting from the last note and ending at the first.

First we observe that this operation reflects every point with respect to some axis. For example, retrogradation maps point (x, y, z) to (z, y, x), this is a reflection with respect to x=z. It similarly maps point (a, b, c, d) to (d, c, b, a), a reflection with respect to $a = d, b = c$. This is a beautiful operation that reminds us of taking inverse of projections in certain planes. Pleasantly, the composite of retrograde, like composite of inverse, maps back to the original point.

Second, we want to observe what happens to the entropy of a piece of music after retrogradation.

Applying the same method of Markov's chain to infinity, we can write a programs for pitch entropy.

ENTROPY (pitch)	Bach Crab Canon	bach bwv846	bach a lovers concerto	bach bwv1006	bach bwv996_4	chopin prelude op.38	chopin nocturne	chopin waltz op.69	haydn reversible menuet	pachelbel canon in D	mozart k618 canon like counterpt	mozart requiem k626
original	0.776876	0.73815	0.785369	0.748559	0.900784	1.773687	1.09434	1.169508	1.040853	0.731689	0.687259	0.904675
retrograde	0.774823	0.741214	0.882845	1.217459	1.227154	0.941102	1.128741	1.053228	1.093163	0.769659	0.693178	0.934072
merged	0.715429	0.743708	0.879226	1.173209	1.103122	0.90085	1.166054	1.119957	1.054764	0.783345	0.704586	0.912962
delta original vs. retrograde(%)	-0.26426	0.415092	12.41149	62.64035	36.23177	-46.94092024	3.143539	-9.94264	5.025686	5.189363	0.861247361	3.249454224
delta original vs. merged (%)	-7.9095	0.752963	11.95069	56.729	22.46243	-49.21031727	6.553174	-4.23691	1.3365	7.05983	2.521174695	0.916019565
delta retrograde vs. merged (%)	-7.66549	0.336475	-0.40992	-3.63462	-10.1073	-4.277113427	3.305718	6.335665	-3.51265	1.77819	1.645753327	-2.259997088

(the chart includes investigations on sheets where the original piece is merged with the retrograded piece to see what happens if the piece is played on a mobius strip)

We notice that for some music piece, the the change in average entropy is relatively small, this includes the famous “Crab Canon”, that can be played on Mobius strips. Its construction ensures that its retrogradation in a large part overlaps with the original melody, which guarantees a limited variation in number of pitch permutations. But this construction is very unique, and we see some other small changes in retrograded piece, so it’s natural to think about what structure in a piece of music makes this ΔH small.

For simplicity, we’ll use a 2D model to investigate. For a graph, where $(i, j) \in \mathbb{R}^2$ refers to all points on the plane, not necessarily belong to the graph,

$$H(X) = -\sum_i \sum_j \frac{\mu_{(i,j)}}{\mu_i} ln \frac{\mu_{(i,j)}}{\mu_i}$$

retrogradation maps point (i, j) to (j, i), and the new graph’s corresponding entropy becomes:

$$H'(X) = -\sum_j \sum_i \frac{\mu_{(j,i)}}{\mu_j} ln \frac{\mu_{(j,i)}}{\mu_j}$$

Note that the number of edges connected to a vertex wouldn't change after reflection, so $\mu_{(j,i)} = \mu_{(i,j)}$. And since we are doing summation over the same lattice, the order of summation doesn't matter for i and j. So

$$H'(X) = -\sum_i \sum_j \frac{\mu_{(i,j)}}{\mu_j} ln \frac{\mu_{(i,j)}}{\mu_j}$$

$$\Delta H = H'(x) - H(x) = -\sum_i \sum_j \frac{\mu_{(i,j)}}{\mu_j} ln \frac{\mu_{(i,j)}}{\mu_j} + \sum_i \sum_j \frac{\mu_{(i,j)}}{\mu_i} ln \frac{\mu_{(i,j)}}{\mu_i}$$

$$= \sum_i \sum_j \left(\frac{\mu_{(i,j)}}{\mu_i} ln \frac{\mu_{(i,j)}}{\mu_i} - \frac{\mu_{(i,j)}}{\mu_j} ln \frac{\mu_{(i,j)}}{\mu_j}\right)$$

$$= \sum_i \sum_j \mu_{(i,j)} \left[ln \mu_{(i,j)} \left(\frac{1}{\mu_j} - \frac{1}{\mu_i}\right) + \left(\frac{1}{\mu_i} ln \frac{1}{\mu_i} - \frac{1}{\mu_j} ln \frac{1}{\mu_j}\right)\right]$$

We recognize the possibility that there exists arrays that cancels each other in the summation, but then the possibility becomes too chaotic. So here we construct our model judging which pieces encodes less ΔH through analyzing how close for each point (i, j) on the plane, how close its corresponding $\mu_i, \mu_j, \mu_{(i,j)}$ is to the solutions of $lnn\left(\frac{1}{y} - \frac{1}{x}\right) + \left(\frac{1}{x} ln \frac{1}{x} - \frac{1}{y} ln \frac{1}{y}\right) = 0;\ n, x, y \in \mathbb{Z}^+$.

Apparently, $\mu_i = \mu_j$ is a trivial solution to this equation. So when the graph of a piece resembles a square matrix, it's guaranteed that it has a small ΔH.

But this relationship is not an if and only if, to improve our accuracy, we define the lattice

consistency error of a point (i, j), now it has to be a part of the graph for validity, as

$$\min(|\mu_{(i,j)} - n| + |\mu_i - x| + |\mu_j - y|)$$

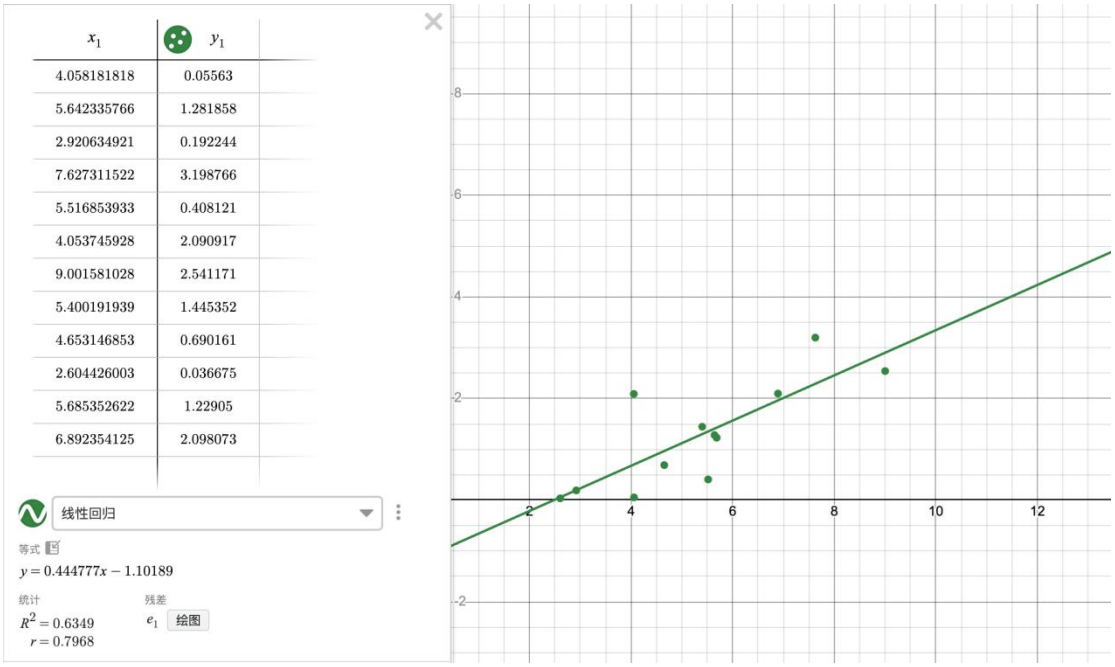
among all (n,x,y) in the solutions set. Take the sum of lattice consistency error of each valid point, we get the total error (the programs written for this part is in Appendix 2).

We test this model on delta in first and second order entropy.

	delta in original vs retrograde	Bach Crab Canon	bach bwv846	bach a lovers concerto	bach bwv1006	bach bwv996_4	chopin prelude op.38	chopin nocturne	chopin waltz op.69	haydn reversible menuet	pachelbel canon in D	mozart k618 canon like counterpt	mozart requiem k626
ord1	ori	20.432617	21.570217	15.903893	26.518249	26.3714	19.176036	28.472245	27.197885	14.117946	16.873691	24.951082	25.659638
	retro	20.488247	20.288359	16.096137	29.717015	25.963279	17.085119	31.013416	28.643237	14.808107	16.837016	26.180132	27.757711
	delta	0.05563	1.281858	0.192244	3.198766	0.408121	2.090917	2.541171	1.445352	0.690161	0.036675	1.22905	2.098073
ord2	ori	34.689801	57.601942	43.447814	77.546247	49.87841	28.231325	85.852034	73.9838	42.970413	59.045274	91.876592	92.325511
	retro	34.595341	61.915344	46.601787	110.972385	80.056828	37.758271	109.721917	72.790975	48.201131	59.626318	90.562609	105.941259
	delta	0.09446	4.313402	3.153973	33.426138	30.178418	9.526946	23.869883	1.192825	5.230718	0.581044	1.313983	13.615748

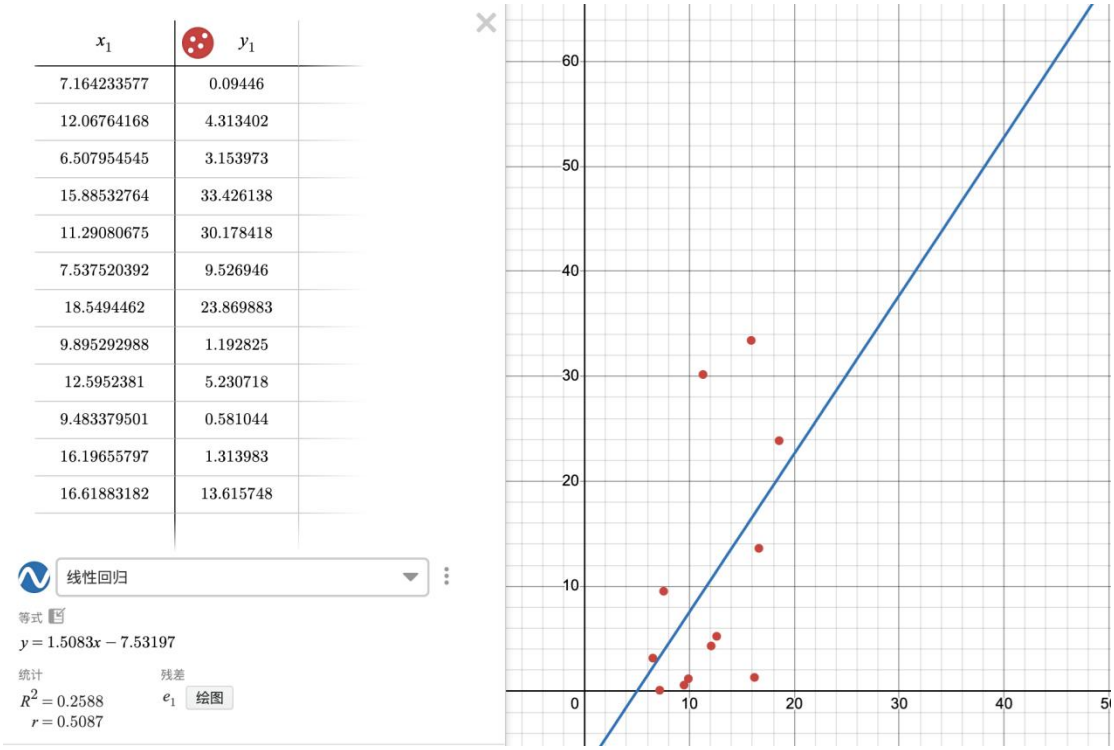
lattice consistency error	Bach Crab Canon	bach bwv846	bach a lovers concerto	bach bwv1006	bach bwv996_4	chopin prelude op.38	chopin nocturne	chopin waltz op.69	haydn reversible menuet	pachelbel canon in D	mozart k618 canon like counterpt	mozart requiem k626
ord1	4.058181818	5.642335766	2.920634921	7.627311522	5.516853933	4.053745928	9.001581028	5.400191939	4.653146853	2.604426003	5.685352622	6.892354125
ord2	7.164233577	12.06764168	6.507954545	15.88532764	11.29080675	7.537520392	18.5494462	9.895292988	12.5952381	9.483379501	16.19655797	16.61883182

The following is the first order lattice consistency error vs. ΔH graph.



x_1	y_1
4.058181818	0.05563
5.642335766	1.281858
2.920634921	0.192244
7.627311522	3.198766
5.516853933	0.408121
4.053745928	2.090917
9.001581028	2.541171
5.400191939	1.445352
4.653146853	0.690161
2.604426003	0.036675
5.685352622	1.22905
6.892354125	2.098073

Statistically, r>0.7 shows a strong linear correlation between points, indicating that our model is a good fit for predicting which music pieces experience less change under operation retrogradation.



x_1	y_1
7.164233577	0.09446
12.06764168	4.313402
6.507954545	3.153973
15.88532764	33.426138
11.29080675	30.178418
7.537520392	9.526946
18.5494462	23.869883
9.895292988	1.192825
12.5952381	5.230718
9.483379501	0.581044
16.19655797	1.313983
16.61883182	13.615748

The second order lattice consistency error vs. ΔH graph also shows a medium linear correlation between the two variables since $0.3<r<0.7$. So although order two entropy introduced more possibilities influencing ΔH, it's still correlated to how close $\mu_i, \mu_j, \mu_{(i,j)}$ are to the solution set.

Finally, other than retrograde, there are a lot of other operations that we can apply to a piece of music. It's natural to think, does all of the operations create a ΔH? or are there some kind of operations that preserves entropy? For example, transposition, shifting the whole melody upward or downward, the graph of this music is simply shifting leftward or rightward, it does not interfere the music's structure and thus wouldn't cause change in entropy. In fact, every bijective function preserves the structure of all pitch permutations. However, operations like compressing octave (taking modulo twelve for all pitches) or permuting the notes would influence the piece's entropy. Interestingly, the octave operation folds the original space into a torus, and imagine the original plane transparent, edges connecting to same note in different octaves merges into one vertex.

Conclusion

This research is still at an initial stage because the investigation of the graph, harmony(entropy+ retrograde), and generalization is not sufficient enough yet. Our observations regarding graphs visualizing music is more focused on locality not the aggregation, and there's a limited investigation on consonance although it plays an important role in the model. There's an observation in regards to consonance and entropy that was

unexplained.

ENTROPY (pitch+rhythm)	Bach Crab Canon	bach bwv846	bach a lovers concerto	bach bwv1006	bach bwv996_4	canon 3 (not Bach!!!)	chopin prelude op.38	chopin nocturne	chopin waltz op.69	haydn reversible menuet	pachelbel canon in D	mozart k618 canon like counterpt	mozart requiem k626	beethoven canon in A flat major	beethoven canon in G major
original	1.314824	0.809712	1.539459	1.309411	1.54423	1.2729	2.063788	1.461123	1.779144	1.802646	1.097662	1.415487	1.625409	0.999572	1.718163
retrograde	1.32778	0.746334	1.721653	1.929654	1.982966	1.31821	1.265856	1.660317	1.506291	1.846175	1.017535	1.490919	1.549055	1.10518	0.856227
merged	1.290634	0.76893	1.712373	0.76893	1.805867	1.279312	1.386781	1.688352	1.676586	1.767789	1.023926	1.427446	1.608102	1.393537	1.796638
delta original vs. retrograde(%)	0.985379032	-7.827227459	11.83493682	47.36809145	28.41131179	3.559588342	-38.66346737	13.6329385	-15.33619538	2.414728127	-7.299788095	5.329049295	-4.697525361	10.56532196	-50.16613674
delta original vs. merged (%)	-1.839789964	-5.036605608	11.23212765	-41.27665034	16.94287768	0.503731636	-32.80409616	15.55166813	-5.764457514	-1.933657523	-6.717550576	0.844868233	-1.064778157	39.41336892	4.567378066
delta retrograde vs. merged (%)	-2.797602012	3.027598903	-0.539016863	-60.15192361	-8.931015459	-2.950819672	9.552824334	1.688532973	11.30558438	-4.245859683	0.628086503	-4.25730707	3.811807844		

CONSONANCE	Bach Crab Canon	bach bwv846	bach a lovers concerto	bach bwv1006	bach bwv996_4	canon 3	chopin prelude op.38	chopin nocturne	chopin waltz op.69	haydn reversible menuet	pachelbel canon in D	mozart k618 canon like counterpt	mozart requiem k626	beethoven canon in A flat major	beethoven canon in G major
original	N/A	N/A	0.669	0.447	0.516	0.71	0.416	0.541	0.525	0.656	0.691	0.633	0.722	0.543	0.784
retrograde			0.611	0.506	0.504	0.578	0.417	0.507	0.482	0.5	0.649	0.626	0.688	0.653	0.665
merged	0.824	0.582	0.381	0.417	0.455	0.379	0.175	0.31	0.249	0.451	0.398	0.359	0.463	0.455	0.459
delta original vs. retrograde(%)	#VALUE!	#VALUE!	-8.669656203	13.19910515	-2.325581395	-18.5915493	0.240384615	-6.284658041	-8.19047619	-23.7804878	-6.078147612	-1.105845182	-4.709141274	20.25782689	-15.17857143
delta original vs. merged (%)	#VALUE!	#VALUE!	-43.04932735	-6.711409396	-11.82170543	-46.61971831	-57.93269231	-42.6987061	-52.57142857	-31.25	-42.40231548	-43.28593997	-35.87257618	-16.20626151	-41.45408163
delta retrograde vs. merged (%)	#DIV/0!	#DIV/0!	-37.64320786	-17.58893281	-9.722222222	-34.42906574	-58.03357314	-38.85601578	-48.34024896	-9.8	-38.67488444	-42.65175719	-32.70348837		

As the result shown, the pieces with change in consonance or entropy within 15% is highlighted, and the pattern in the chart seems to be complementary, they seem to be inversely proportional. However, this result that might have crucial influence on our definition and measurement of harmony is not yet proven to be true nor synchronized. At last, though this model of visualization and many conclusions in this paper is perceivably generalizable to higher dimensions and other quotient spaces, to investigate more music objects like voice leading and chord class, this paper did not provide full derivation and unfold the whole picture.

To sum up, this paper explored a mathematical lens on musical harmony by defining it as a union of information entropy and consonance. Through visualization of music, we developed

a framework using graph theory to analyze entropy. It also provides a model to describe, or qualitatively estimate, the entropy via the graph's shape and the closeness to a given solution set. The discovery reflects a connection between mathematical abstraction and musical perception, and can be continued through making holistic observation of graphs of visualized music, incorporating consonance more, and further generalizing the model.

References

Callender, C., I. Quinn, and D. Tymoczko. 2008. “Generalized Voice-Leading Spaces.” Science 320 (5874): 346–48. https://doi.org/10.1126/science.1153021.

D Gareth Loy, John Chowning, and Inc Ebrary. 2006. Musimathics : The Mathematical Foundations of Music. Cambridge, Mass.: Mit Press.

G Mazzola. 2013. Topos of Music : Geometric Logic of Concepts, Theory, and Performance. Birkhauser.

Huron, David. 2006. Sweet Anticipation : Music and the Psychology of Expectation. Erscheinungsort Nicht Ermittelbar: Mit Press.

Sethares, William A. 2005. Tuning, Timbre, Spectrum, Scale. London: Springer.

Appendix 1

Average Pitch Entropy / Total Entropy:

```python
from music21 import converter, note, chord
from collections import defaultdict
import math

# Load the score (adjust path to your file)
score = converter.parse('chopin prelude op.38_retrograde.mid')  # Replace with your file's path

# Extract all notes and chords from the score
# Count transitions for each note in chords but count chords as one note in total note count
elements = []  # Holds the sequence of notes (including all individual pitches from chords)
total_notes = 0  # Counts each note or chord as one for total note calculation

# Iterate through all notes and chords in the score
for element in score.recurse().notes:
    if isinstance(element, chord.Chord):
        for pitch in element.pitches:
            elements.append(pitch.name)  # Add each pitch in the chord to elements
        total_notes += 1  # Count the chord as one note
    elif isinstance(element, note.Note):
        elements.append(element.name)  # Add single note to elements
        total_notes += 1

# Function to calculate entropy for a given order
def calculate_entropy(order, sequence):  1个用法
    # Create dictionaries to count occurrences of sequences and their continuations
    counts = defaultdict(int)
    total_context_counts = defaultdict(int)

    # Slide through the sequence to count occurrences
    for i in range(len(sequence) - order):
        context = tuple(sequence[i:i + order])  # Previous notes as context
        next_note = sequence[i + order]  # Next note to be predicted
        counts[(context, next_note)] += 1
        total_context_counts[context] += 1

    # Compute entropy for this order
    entropy = 0
    for (context, next_note), count in counts.items():
```

```python
    entropy = calculate_entropy(order, elements)

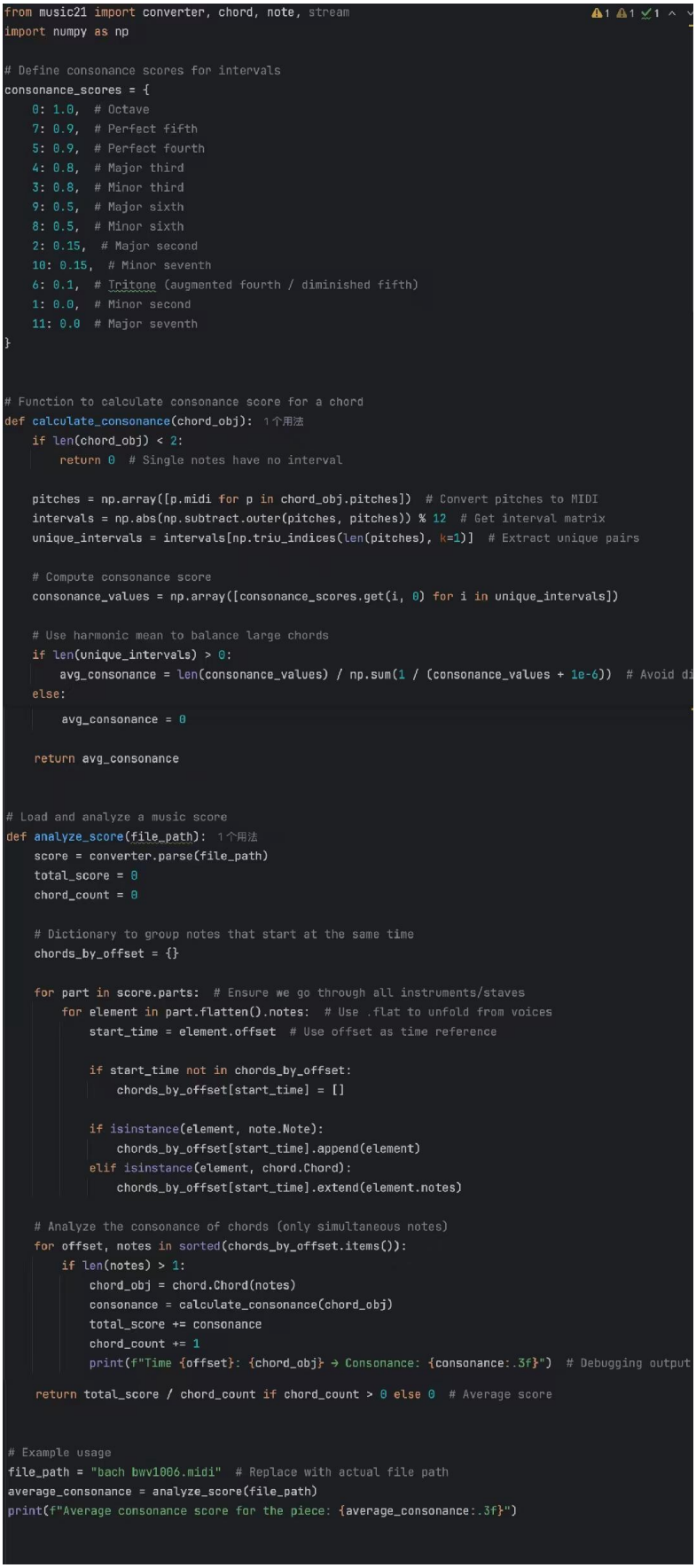
    if stop_after_zero:  # Reduce the counter for extra orders
        additional_orders_after_zero -= 1
        if additional_orders_after_zero < 0:  # Stop after additional orders
            break

    if entropy == 0 and not stop_after_zero:
        stop_after_zero = True  # Trigger the extra order loop

    total_entropy += entropy
    print(f"Order {order} entropy: {entropy:.6f}")

# Compute average entropy per note (chords treated as one note here)
average_entropy = total_entropy / total_notes
print(f"Total entropy (all orders): {total_entropy:.6f}")
print(f"Average entropy per note: {average_entropy:.6f}")
```

Average Consonance Score:



```python
from music21 import converter, chord, note, stream
import numpy as np

# Define consonance scores for intervals
consonance_scores = {
    0: 1.0,  # Octave
    7: 0.9,  # Perfect fifth
    5: 0.9,  # Perfect fourth
    4: 0.8,  # Major third
    3: 0.8,  # Minor third
    9: 0.5,  # Major sixth
    8: 0.5,  # Minor sixth
    2: 0.15,  # Major second
    10: 0.15,  # Minor seventh
    6: 0.1,  # Tritone (augmented fourth / diminished fifth)
    1: 0.0,  # Minor second
    11: 0.0  # Major seventh
}


# Function to calculate consonance score for a chord
def calculate_consonance(chord_obj):  1个用法
    if len(chord_obj) < 2:
        return 0  # Single notes have no interval

    pitches = np.array([p.midi for p in chord_obj.pitches])  # Convert pitches to MIDI
    intervals = np.abs(np.subtract.outer(pitches, pitches)) % 12  # Get interval matrix
    unique_intervals = intervals[np.triu_indices(len(pitches), k=1)]  # Extract unique pairs

    # Compute consonance score
    consonance_values = np.array([consonance_scores.get(i, 0) for i in unique_intervals])

    # Use harmonic mean to balance large chords
    if len(unique_intervals) > 0:
        avg_consonance = len(consonance_values) / np.sum(1 / (consonance_values + 1e-6))  # Avoid di
    else:
        avg_consonance = 0

    return avg_consonance


# Load and analyze a music score
def analyze_score(file_path):  1个用法
    score = converter.parse(file_path)
    total_score = 0
    chord_count = 0

    # Dictionary to group notes that start at the same time
    chords_by_offset = {}

    for part in score.parts:  # Ensure we go through all instruments/staves
        for element in part.flatten().notes:  # Use .flat to unfold from voices
            start_time = element.offset  # Use offset as time reference

            if start_time not in chords_by_offset:
                chords_by_offset[start_time] = []

            if isinstance(element, note.Note):
                chords_by_offset[start_time].append(element)
            elif isinstance(element, chord.Chord):
                chords_by_offset[start_time].extend(element.notes)

    # Analyze the consonance of chords (only simultaneous notes)
    for offset, notes in sorted(chords_by_offset.items()):
        if len(notes) > 1:
            chord_obj = chord.Chord(notes)
            consonance = calculate_consonance(chord_obj)
            total_score += consonance
            chord_count += 1
            print(f"Time {offset}: {chord_obj} → Consonance: {consonance:.3f}")  # Debugging output

    return total_score / chord_count if chord_count > 0 else 0  # Average score


# Example usage
file_path = "bach bwv1006.midi"  # Replace with actual file path
average_consonance = analyze_score(file_path)
print(f"Average consonance score for the piece: {average_consonance:.3f}")
```

Appendix 2

Solution Set:

```python
import math
import os


def find_zero_solutions(range_n, range_xy, epsilon=1e-6):  1个用法
    solutions = []

    for n in range_n:
        if n <= 0:
            continue
        a = math.log(n)
        for x in range_xy:
            for y in range_xy:
                if x <= 0 or y <= 0:
                    continue

                term1 = a * (1 / y - 1 / x)
                term2 = (1 / x * math.log(1 / x) - 1 / y * math.log(1 / y))
                b = term1 + term2

                if abs(b) < epsilon:
                    solutions.append((x, y, n))  # (x, y, n) format for compatibility

    return solutions


# Define the ranges
range_n = range(1, 51)  # Increase range to find more precise matches
range_xy = range(1, 101)  # Try x, y = 1 to 100

solutions = find_zero_solutions(range_n, range_xy)

# Write output to a file in a format compatible with your program
filename = "solutions.txt"
filepath = os.path.abspath(filename)
print(f"Saving results to: {filepath}")

with open(filename, "w", encoding="gb18030") as f:
    if solutions:
        for x, y, n in solutions:
            f.write(f"({x},{y},{n})\n")
    else:
        f.write("No solutions found.\n")

print("✅ Done! Solutions where b ≈ 0 have been saved.")
```

Point Set 2D:

```python
from music21 import converter, note, chord, pitch
import os

# Function to convert pitch to piano key number (A0 = 1, C8 = 88)
def pitch_to_key_number(p):  2 用法
    return p.midi - 20  # MIDI 21 (A0) → key 1

# Load the score (adjust the path to your file)
score = converter.parse('mozart requiem k.626.midi')  # Replace with your MIDI file path

# Extract all pitches (including from chords)
all_keys = []

for element in score.recurse().notes:
    if isinstance(element, chord.Chord):
        for p in element.pitches:
            key_num = pitch_to_key_number(p)
            if 1 <= key_num <= 88:
                all_keys.append(key_num)
    elif isinstance(element, note.Note):
        key_num = pitch_to_key_number(element.pitch)
        if 1 <= key_num <= 88:
            all_keys.append(key_num)

# Generate transitions: (note1, note2)
transitions = []
for i in range(len(all_keys) - 1):
    transitions.append((all_keys[i], all_keys[i + 1]))

# Write transitions to a file
output_file = "note_transitions_1.txt"
with open(output_file, "w") as f:
    for a, b in transitions:
        f.write(f"({a},{b})\n")

print(f"Saved {len(transitions)} transitions to '{os.path.abspath(output_file)}'")
```

Point Set 3D:

```python
from music21 import converter, note, chord
import datetime

# Convert pitch to piano key number (A0=1, C8=88)
def pitch_to_piano_key(p):  2 用法
    midi = p.midi
    return midi - 20  # A0 is MIDI 21 → piano key 1

# Load the score
score = converter.parse('mozart requiem k.626.midi')  # Replace with your file path

# Extract all individual pitches (expanding chords)
pitches = []

for element in score.recurse().notes:
    if isinstance(element, chord.Chord):
        pitches.extend([pitch_to_piano_key(p) for p in element.pitches])
    elif isinstance(element, note.Note):
        pitches.append(pitch_to_piano_key(element.pitch))

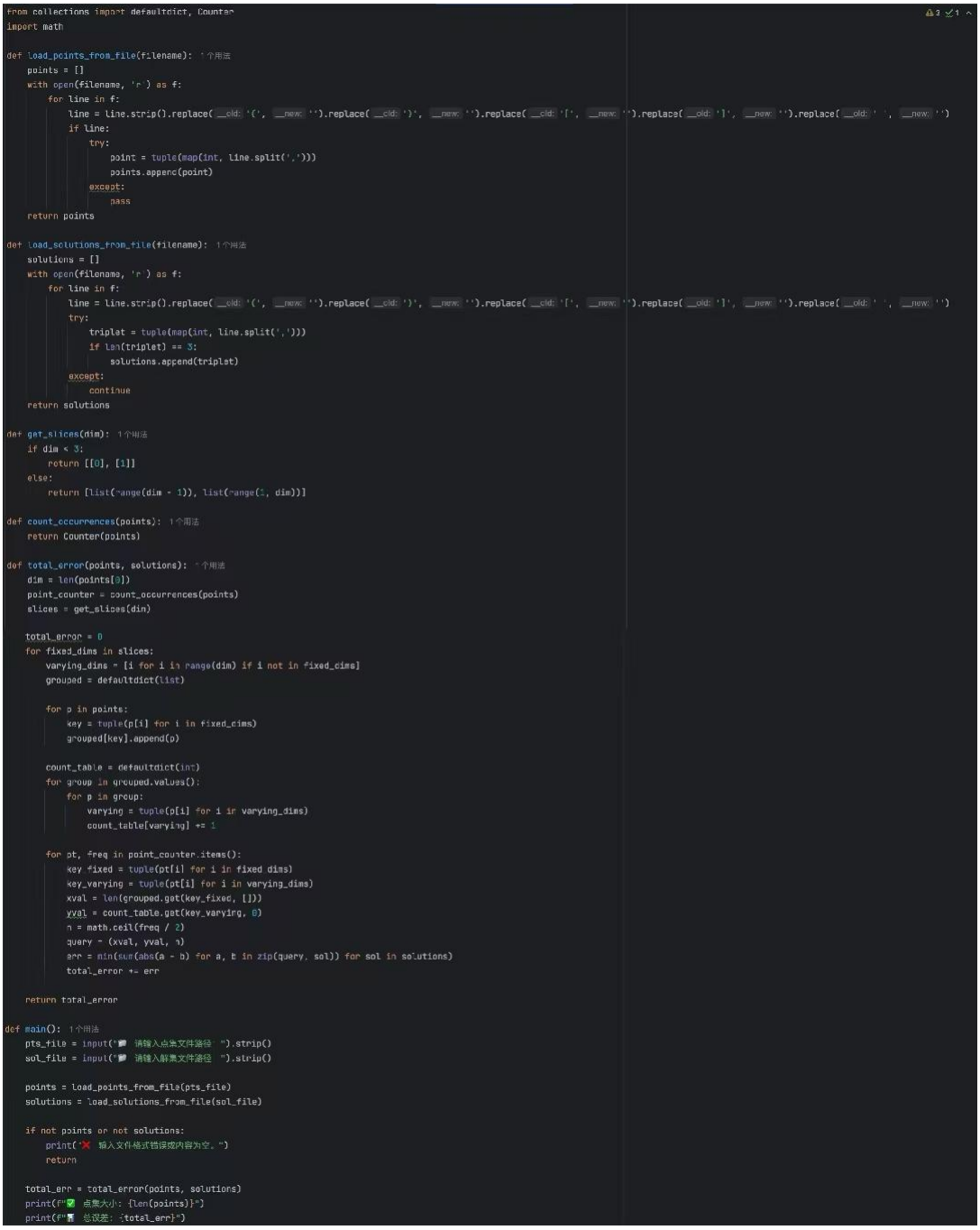
# Record 3-note transitions
triplets = []
for i in range(len(pitches) - 2):
    triplet = (pitches[i], pitches[i + 1], pitches[i + 2])
    triplets.append(triplet)

# Generate output file with timestamp
timestamp = datetime.datetime.now().strftime("%Y%m%d_%H%M%S")
output_file = f"note_transitions_2.txt"

# Write to file
with open(output_file, "w") as f:
    for triplet in triplets:
        f.write(f"{triplet}\n")

print(f"Saved {len(triplets)} 3-note transitions to {output_file}")
```

Lattice Consistency Error General:



```python
from collections import defaultdict, Counter
import math

def load_points_from_file(filename):  1 个用法
    points = []
    with open(filename, 'r') as f:
        for line in f:
            line = line.strip().replace('(', '').replace(')', '').replace('[', '').replace(']', '').replace(' ', '')
            if line:
                try:
                    point = tuple(map(int, line.split(',')))
                    points.append(point)
                except:
                    pass
    return points

def load_solutions_from_file(filename):  1 个用法
    solutions = []
    with open(filename, 'r') as f:
        for line in f:
            line = line.strip().replace('(', '').replace(')', '').replace('[', '').replace(']', '').replace(' ', '')
            try:
                triplet = tuple(map(int, line.split(',')))
                if len(triplet) == 3:
                    solutions.append(triplet)
            except:
                continue
    return solutions

def get_slices(dim):  1 个用法
    if dim < 3:
        return [[0], [1]]
    else:
        return [list(range(dim - 1)), list(range(1, dim))]

def count_occurrences(points):  1 个用法
    return Counter(points)

def total_error(points, solutions):  1 个用法
    dim = len(points[0])
    point_counter = count_occurrences(points)
    slices = get_slices(dim)

    total_error = 0
    for fixed_dims in slices:
        varying_dims = [i for i in range(dim) if i not in fixed_dims]
        grouped = defaultdict(list)

        for p in points:
            key = tuple(p[i] for i in fixed_dims)
            grouped[key].append(p)

        count_table = defaultdict(int)
        for group in grouped.values():
            for p in group:
                varying = tuple(p[i] for i in varying_dims)
                count_table[varying] += 1

        for pt, freq in point_counter.items():
            key_fixed = tuple(pt[i] for i in fixed_dims)
            key_varying = tuple(pt[i] for i in varying_dims)
            xval = len(grouped.get(key_fixed, []))
            yval = count_table.get(key_varying, 0)
            n = math.ceil(freq / 2)
            query = (xval, yval, n)
            err = min(sum(abs(a - b) for a, b in zip(query, sol)) for sol in solutions)
            total_error += err

    return total_error

def main():  1 个用法
    pts_file = input("📂 请输入点集文件路径：").strip()
    sol_file = input("📂 请输入解集文件路径：").strip()

    points = load_points_from_file(pts_file)
    solutions = load_solutions_from_file(sol_file)

    if not points or not solutions:
        print("❌ 输入文件格式错误或内容为空。")
        return

    total_err = total_error(points, solutions)
    print(f"✅ 点集大小：{len(points)}")
    print(f"📉 总误差：{total_err}")
```